
type: skill-library

Genre: sop

title: 'Obsidian Skill Library - All in One'

skills: 58

tags:

- skill
- skill-library
- obsidian

aliases:

- 'All Skills'
 - 'Skill Library All in One'
-

Obsidian Skill Library — All in One

All **58 skills** merged into a single document, grouped by category. Every skill keeps its trigger phrases, aliases, and step-by-step procedure.

How to use: jump via the contents below, or search the file — every skill name, alias, and trigger phrase is plain text, so Ctrl/Cmd+F finds a skill by whatever you'd call it. To go back to the per-note vault layout instead, use the zip package.

***Note on tags:** a single markdown file can only carry one YAML frontmatter block, so the per-skill #skill/<category> tags appear inline in each skill's metadata block. The zip version keeps full frontmatter per note for tag-based indexing.*

Contents

- **Meta / Index** (1) — [Skill Canvas](#)
- **Research** (5) — [Account Research](#), [Competitive Brief](#), [Contact Research](#), [Customer Research](#), [Deep Research](#)
- **Writing** (8) — [Ad Copy](#), [Brand Voice Enforcement](#), [Content Creation](#), [Draft Response](#), [Job Description](#), [Process Documentation](#), [Status Report](#), [UX Copy](#)
- **Data** (3) — [Analyze](#), [Create Visualization](#), [Explore Data](#)
- **Finance** (5) — [Budget Optimization](#), [Cash Flow Snapshot](#), [LTV:CAC](#), [Sales Forecast](#), [Variance Analysis](#)
- **Marketing** (6) — [Attribution Report](#), [Campaign Audit](#), [Campaign Plan](#), [Email Sequence](#), [Performance Report](#), [SEO Audit](#)
- **Integration** (5) — [Data App](#), [GitHub Coding](#), [Google Cloud](#), [Manus API Integration](#), [Payments](#)
- **QA / Testing** (5) — [Accessibility Review](#), [Bug Triage](#), [Quality Metrics](#), [Regression Suite](#), [Test Strategy](#)

- **HR / People** (5) — [Interview Kit](#), [Offer Analysis](#), [Onboarding](#), [People Report](#), [Performance Review](#)
 - **Engineering** (5) — [Architecture Decision](#), [Code Review](#), [Debugging](#), [Incident Response](#), [Supply Chain Security](#)
 - **Productivity** (4) — [Call Preparation](#), [Digest](#), [Standup](#), [Task Management](#)
 - **Media** (3) — [Design Critique](#), [Media Generation](#), [Video Edit](#)
 - **Legal** (3) — [Compliance Tracking](#), [Contract Review](#), [Risk Assessment](#)
-

Meta / Index

Skill Canvas

- **Type** — skill
- **Category** — meta #skill/meta
- **Trigger** — overview, index, map of skills
- **Aliases** — Skill Canvas, skill map, skill index, obsidian skills
- **Note** — skill-canvas.md (in the zip package)

The master index note that maps every skill in the vault by tag, type, and alias.

Procedure

This is the hub. Use Obsidian tags (#skill, #skill/<category>) and Dataview to query the library.

- Keep this note linked from the vault home.
 - Every skill note below inherits type: skill and its category tag.
 - Run a Dataview query to render the full table of skills.
-

Research

Account Research

- **Type** — skill
- **Category** — research #skill/research
- **Trigger** — research a company, tell me about an account, what's going on with a company, account overview
- **Aliases** — company research, account overview, account fit
- **Note** — account-research.md (in the zip package)

Research a company using CRM and signal data to build an account overview and fit assessment.

Procedure

1. Pull CRM records for the account.
 2. Gather recent signals (email, calls, deals, engagement).
 3. Compile a profile: fit, momentum, risks, and next best action.
-

Competitive Brief

- **Type** — skill
- **Category** — research #skill/research
- **Trigger** — competitive analysis, how do we compare to, competitor research, battlecard
- **Aliases** — competitor analysis, battlecard, positioning gap
- **Note** — competitive-brief.md (in the zip package)

Research competitors and generate positioning and messaging comparison with gaps and opportunities.

Procedure

1. Identify the competitor set.
 2. Research each competitor's positioning, messaging, strengths, gaps.
 3. Build a comparison matrix and battlecard.
 4. Surface unclaimed positioning angles.
-

Contact Research

- **Type** — skill
- **Category** — research #skill/research
- **Trigger** — who is this person, look up a contact, research a contact, contact profile
- **Aliases** — person research, contact profile, lead lookup
- **Note** — contact-research.md (in the zip package)

Research a specific person using CRM and signal data; build a contact profile.

Procedure

1. Resolve the contact in CRM.
 2. Gather role, history, engagement signals.
 3. Produce a profile with warmth assessment and outreach angle.
-

Customer Research

- **Type** — skill
- **Category** — research #skill/research
- **Trigger** — research a customer question, look this up, has this bug been reported before, gather background
- **Aliases** — customer lookup, support research, bug reported before
- **Note** — customer-research.md (in the zip package)

Multi-source research on a customer question or topic with source attribution.

Procedure

1. Search support history, docs, and connected sources.
2. Check whether this was previously reported/told.
3. Summarize with attribution and a recommended response path.

Deep Research

- **Type** — skill
- **Category** — research #skill/research
- **Trigger** — compare the top, X vs Y vs Z, competitor comparison, best options for
- **Aliases** — research, comparison research, multi-source
- **Note** — deep-research.md (in the zip package)

Parallel, cited research across several independent threads, then synthesize a single comparison with sources.

Procedure

1. Split the question into independent subtopics/entities.
 2. Fan out one search per subtopic in parallel.
 3. Collect source excerpts and URLs.
 4. Synthesize into a single comparison with inline citations.
 5. Flag gaps and conflicting evidence.
-

Writing

Ad Copy

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — write ad copy, write headlines for, generate ad variations
- **Aliases** — google ads copy, meta ads copy, headline variations
- **Note** — ad-copy.md (in the zip package)

Write high-converting ad copy for Google Search, PMAX, or Meta campaigns.

Procedure

1. Gather campaign goal, audience, offers.
 2. Draft headlines and descriptions.
 3. Follow platform character limits.
 4. Provide multiple variations for testing.
-

Brand Voice Enforcement

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — write in our brand voice, check this against brand voice, make this sound like us
- **Aliases** — brand voice, on-brand copy, tone check
- **Note** — brand-voice-enforcement.md (in the zip package)

Apply brand voice guidelines to any AI-generated content.

Procedure

1. Load the brand voice guidelines.
 2. Apply them to the content.
 3. Check tone, terminology, and claims.
 4. Deliver on-brand copy.
-

Content Creation

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — write marketing content, content creation, draft a blog post
- **Aliases** — marketing copy, blog post, social post, newsletter
- **Note** — content-creation.md (in the zip package)

Draft marketing content across channels — blog, social, email, landing pages, press releases, case studies.

Procedure

1. Confirm channel and audience.
 2. Draft headline options and body.
 3. Format for the specific channel.
 4. Add SEO and CTA.
 5. Offer variations.
-

Draft Response

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — draft a customer response, reply to this customer, respond to an escalation
- **Aliases** — customer reply, escalation response, bad news reply
- **Note** — draft-response.md (in the zip package)

Draft a professional customer-facing response tailored to the situation and relationship.

Procedure

1. Understand the situation and relationship.
 2. Draft a response that matches the context (answer, escalation, bad news, decline).
 3. Keep tone professional and specific.
 4. Flag anything needing approval.
-

Job Description

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — write a job description, job posting for, JD for

- **Aliases** — job posting, role description, hiring copy
- **Note** — job-description.md (in the zip package)

Write compelling, bias-reduced job descriptions with a full posting and social media version.

Procedure

1. Gather role details, responsibilities, requirements.
 2. Write an inclusive, bias-reduced JD.
 3. Produce a social-media version.
 4. Optional ATS publish.
-

Process Documentation

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — document this process, write an SOP, RACI matrix, process documentation
- **Aliases** — SOP, RACI, standard operating procedure
- **Note** — process-doc.md (in the zip package)

Document a business process — flowcharts, RACI, and SOPs.

Procedure

1. Map the process steps and owners.
 2. Build a RACI matrix.
 3. Write the SOP including exceptions and edge cases.
-

Status Report

- **Type** — skill
- **Category** — writing #skill/writing
- **Trigger** — status report, weekly update, project status, leadership update
- **Aliases** — project status, weekly report, kpi report
- **Note** — status-report.md (in the zip package)

Generate a status report with KPIs, risks, and action items.

Procedure

1. Gather project metrics and tracker activity.
 2. Assess green/yellow/red status.
 3. Surface risks and decisions needed.
 4. Write a readable narrative.
-

UX Copy

- **Type** — skill
- **Category** — writing #skill/writing

- **Trigger** — write copy for, what should this button say, review this error message, name a CTA
- **Aliases** — microcopy, error message copy, button label, empty state
- **Note** — ux-copy.md (in the zip package)

Write or review UX copy — microcopy, error messages, empty states, CTAs.

Procedure

1. Identify the UI moment.
2. Write clear, human, action-oriented copy.
3. Match the product voice.
4. Review for clarity and tone.

Data

Analyze

- **Type** — skill
- **Category** — data #skill/data
- **Trigger** — analyze, what does the data say, how many, show me the trend, run an analysis
- **Aliases** — data analysis, data question, trend analysis
- **Note** — analyze.md (in the zip package)

Answer data questions from quick lookups to full analyses.

Procedure

1. Clarify the question and the data source.
2. Query/load the data.
3. Run the analysis (counts, trends, breakdowns).
4. Report with the key finding first.

Create Visualization

- **Type** — skill
- **Category** — data #skill/data
- **Trigger** — create a chart, plot this data, visualize the distribution, bar chart
- **Aliases** — chart, data plot, visualization
- **Note** — create-viz.md (in the zip package)

Create publication-quality data visualizations with Python.

Procedure

1. Understand the data and the story.
2. Choose the right chart type.
3. Generate a clean, labeled figure.
4. Export for the target medium.

Explore Data

- **Type** — skill
- **Category** — data #skill/data
- **Trigger** — explore this dataset, what's in this table, data profiling
- **Aliases** — data profiling, dataset exploration, quality check
- **Note** — explore-data.md (in the zip package)

Profile and explore a dataset to understand its shape, quality, and patterns.

Procedure

1. Load the dataset.
 2. Profile shape, types, nulls, distributions.
 3. Flag quality issues and patterns.
 4. Summarize findings.
-

Finance

Budget Optimization

- **Type** — skill
- **Category** — finance #skill/finance
- **Trigger** — optimize my budget, reallocate budget, where should I spend more, shift budget
- **Aliases** — budget reallocation, campaign budget, spend optimization
- **Note** — budget-optimization.md (in the zip package)

Optimize budget allocation across campaigns based on performance data. Shifts budget from underperformers to top campaigns with a clear before/after.

Procedure

1. Pull campaign performance (spend, ROAS/CPA, conversions) from the connected source.
 2. Rank campaigns by efficiency (ROAS or CPA) and by volume.
 3. Identify underperformers (efficiency below target) and over-performers (efficiency above target with headroom).
 4. Propose reallocation: shift X% from low performers to top performers.
 5. Respect hard caps and minimum spends; never zero out a test campaign without flagging it.
 6. Output a reallocation table with expected impact. Get approval before applying.
-

Cash Flow Snapshot

- **Type** — skill
- **Category** — finance #skill/finance
- **Trigger** — forecast my cash flow, will I make payroll, runway, cash crunch
- **Aliases** — cash forecast, runway, payroll forecast

- **Note** — cash-flow-snapshot.md (in the zip package)

Produce a 30/60/90-day cash flow forecast with confidence bands and named risk flags.

Procedure

1. Read AR/AP and historical cash timing.
 2. Add known fixed costs.
 3. Build 30/60/90-day forecast with variance bands.
 4. Flag named risks.
-

LTV:CAC

- **Type** — skill
- **Category** — finance #skill/finance
- **Trigger** — LTV CAC, unit economics, payback period, customer lifetime value
- **Aliases** — unit economics, payback period, customer economics
- **Note** — ltv-cac.md (in the zip package)

Calculate LTV:CAC ratio, payback period, and unit economics by segment.

Procedure

1. Gather revenue, cost, and cohort data.
 2. Compute LTV, CAC, and ratio by segment.
 3. Compute payback period.
 4. Recommend where to invest vs optimize.
-

Sales Forecast

- **Type** — skill
- **Category** — finance #skill/finance
- **Trigger** — sales forecast, forecast this quarter, gap to quota, pipeline coverage
- **Aliases** — weighted forecast, commit vs upside, quota gap
- **Note** — forecast.md (in the zip package)

Generate a weighted sales forecast with best/likely/worst scenarios and gap analysis.

Procedure

1. Load pipeline CSV.
 2. Apply stage-weighted probabilities.
 3. Build best/likely/worst scenarios.
 4. Split commit vs upside.
 5. Report gap to quota.
-

Variance Analysis

- **Type** — skill

- **Category** — finance #skill/finance
- **Trigger** — budget vs actual, variance analysis, period-over-period changes
- **Aliases** — budget variance, flux analysis, waterfall
- **Note** — variance-analysis.md (in the zip package)

Decompose financial variances into drivers with narrative explanations and waterfall analysis.

Procedure

1. Pull actuals vs budget/prior period.
 2. Isolate drivers by line.
 3. Build a waterfall of the variance.
 4. Write narrative commentary.
-

Marketing

Attribution Report

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — attribution analysis, which channels drive revenue, ROI by channel
- **Aliases** — channel attribution, marketing roi, budget by channel
- **Note** — attribution-report.md (in the zip package)

Analyze marketing attribution across models and identify which channels drive the most valuable users.

Procedure

1. Load attribution data across models.
 2. Compare channel contribution to valuable users.
 3. Recommend budget reallocation.
-

Campaign Audit

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — audit my campaigns, check my ads, what's wrong with my campaigns
- **Aliases** — campaign health, ad audit, quick wins
- **Note** — campaign-audit.md (in the zip package)

Audit all active campaigns for issues, inefficiencies, and quick wins.

Procedure

1. Pull all active campaigns.
2. Check structural issues, targeting, budget.
3. Rank inefficiencies.
4. Surface quick wins before waste.

Campaign Plan

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — campaign plan, plan a product launch, lead-gen campaign, content calendar
- **Aliases** — campaign brief, product launch plan, content calendar
- **Note** — campaign-plan.md (in the zip package)

Generate a full campaign brief with objectives, audience, messaging, channel strategy, and content calendar.

Procedure

1. Confirm objective and audience.
2. Draft messaging and channel strategy.
3. Build a week-by-week calendar with dependencies.
4. Define success metrics.

Email Sequence

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — email sequence, build a drip campaign, lead nurture flow, win-back sequence
- **Aliases** — drip campaign, lead nurture, onboarding emails, win-back
- **Note** — email-sequence.md (in the zip package)

Design and draft multi-email sequences with full copy, timing, branching, and exit conditions.

Procedure

1. Define the goal and audience.
2. Map the email steps and timing.
3. Draft full copy with branching and exit conditions.
4. Add A/B test suggestions.

Performance Report

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — performance report, campaign results, channel summary
- **Aliases** — marketing report, metrics summary, optimization priorities
- **Note** — performance-report.md (in the zip package)

Build a marketing performance report with key metrics, trends, wins, and prioritized recommendations.

Procedure

1. Gather campaign metrics.

2. Compute trends and compare to goals.
 3. Summarize wins and misses.
 4. Recommend prioritized next-period actions.
-

SEO Audit

- **Type** — skill
- **Category** — marketing #skill/marketing
- **Trigger** — run an SEO audit, keyword research for my site, technical SEO check
- **Aliases** — keyword research, on-page seo, content gap
- **Note** — seo-audit.md (in the zip package)

Run a comprehensive SEO audit — keywords, on-page, content gaps, technical checks, competitors.

Procedure

1. Assess site SEO health.
 2. Research keywords and content gaps.
 3. Run technical checks.
 4. Compare to competitors.
 5. Produce a prioritized action plan.
-

Integration

Data App

- **Type** — skill
- **Category** — integration #skill/integration
- **Trigger** — admin panel, internal tool, crud app, editable dashboard
- **Aliases** — dashboard, admin panel, crud tool
- **Note** — data-app.md (in the zip package)

Build, edit, and publish data dashboards, admin panels, and internal CRUD tools with React and charts.

Procedure

1. Clarify the records and actions.
 2. Scaffold a React app with a data table.
 3. Add create/edit/delete and login if needed.
 4. Publish and verify.
-

GitHub Coding

- **Type** — skill
- **Category** — integration #skill/integration

- **Trigger** — clone, git push, pull request, open a PR, github repo
- **Aliases** — git, pull request, github, repo work
- **Note** — github-coding.md (in the zip package)

Read or work on GitHub repos — answer questions with gh, clone, edit, test, push, open PRs.

Procedure

1. Use native git/gh CLI.
 2. Clone/edit/test/push as needed.
 3. Open PRs and respond to review.
 4. Never route GitHub through a connector.
-

Google Cloud

- **Type** — skill
- **Category** — integration #skill/integration
- **Trigger** — gcloud, google cloud, gcp, cloud run, bigquery, firebase, firestore
- **Aliases** — gcp, bigquery, firebase, cloud run, gcs
- **Note** — google-cloud.md (in the zip package)

Work with Google Cloud through gcloud/goutil/bq CLIs — list, inspect, deploy, or change projects and services.

Procedure

1. Ensure GCP provider access.
 2. Use the appropriate CLI (gcloud/goutil/bq).
 3. Inspect before changing.
 4. Verify the result.
-

Manus API Integration

- **Type** — skill
- **Category** — integration #skill/integration
- **Trigger** — use the Manus API, create a Manus task, run a Manus agent, manus
- **Aliases** — manus agent, manus task, agent api, auto-create skills
- **Note** — manus-api.md (in the zip package)

Create, run, and manage Manus AI agent tasks via the Manus API v2 — task lifecycle, file attachments, webhooks, connectors, skills, and structured output.

Procedure

1. Read the API key from MANUS_API_KEY (never hardcode or echo it).
2. Authenticate per Manus API v2 docs.
3. Create a task via task.create with optional files, connectors, skills, agent profile, and structured-output schema.
4. Drive the lifecycle: poll task.listMessages; handle running/stopped/error states.

5. Respond to every waiting event — `sendMessage` for questions, `confirmAction` for confirmations including browser selection.
6. Attach large files via two-step `file.upload` → PUT (20 MB inline cap).
7. Prefer `webhook.create` for push notifications over polling.
8. Parse structured JSON output and return it cleanly.

Auto-create-skills extension: a Manus task can be instructed to author a skill file in this same format and return it, which you then save and register in the Skill Canvas.

Payments

- **Type** — skill
- **Category** — integration #skill/integration
- **Trigger** — add payments, accept payments, sell on my site, checkout, stripe, shopify
- **Aliases** — stripe, shopify, checkout, buy button
- **Note** — payments.md (in the zip package)

Add working payments to a website using the user's own Stripe or Shopify account.

Procedure

1. Stripe: mint a hosted Payment Link. Shopify: connect the store via OAuth.
 2. Stock products/discounts as needed.
 3. Render a live storefront with hosted checkout.
 4. No secret keys ever touch the site.
-

QA / Testing

Accessibility Review

- **Type** — skill
- **Category** — qa #skill/qa
- **Trigger** — audit accessibility, check a11y, is this accessible, WCAG audit
- **Aliases** — a11y, wcag, accessibility audit
- **Note** — accessibility-review.md (in the zip package)

Run a WCAG 2.1 AA accessibility audit on a design or page.

Procedure

1. Check color contrast, keyboard navigation, focus order, and touch-target size.
 2. Review semantics and screen-reader behaviour.
 3. Flag each failure against its WCAG criterion.
 4. Prioritize by real user impact, not by checklist order.
 5. Verify the fixes, do not assume them.
-

Bug Triage

- **Type** — skill
- **Category** — qa #skill/qa
- **Trigger** — triage this bug, prioritize defects, how severe is this bug, bug triage
- **Aliases** — defect triage, bug priority, severity assessment
- **Note** — bug-triage.md (in the zip package)

Triage and prioritize defects by severity, impact, and reproducibility, then route them to an owner.

Procedure

1. Reproduce and capture exact steps plus environment.
 2. Classify severity (impact) separately from priority (urgency).
 3. Check for duplicates and known issues before creating new work.
 4. Assign an owner and a target release.
 5. Record the decision so the same bug is not re-triaged.
-

Quality Metrics

- **Type** — skill
- **Category** — qa #skill/qa
- **Trigger** — quality metrics, defect density, coverage report, escape rate
- **Aliases** — qa metrics, defect metrics, test coverage report
- **Note** — quality-metrics.md (in the zip package)

Track the quality signals that drive decisions — defect density, escape rate, coverage, flake rate.

Procedure

1. Define the few metrics that actually drive decisions.
 2. Pull them from the tracker and CI, not by hand.
 3. Trend them over time — a point-in-time number decides nothing.
 4. Watch the leading indicator (flake rate, review latency) before the lagging one (escaped defects).
 5. Report every metric with the action it implies.
-

Regression Suite

- **Type** — skill
- **Category** — qa #skill/qa
- **Trigger** — regression suite, regression test, what should we re-test, prevent regressions
- **Aliases** — regression testing, smoke suite, re-test plan
- **Note** — regression-suite.md (in the zip package)

Build and maintain a regression suite that catches breakage without slowing delivery.

Procedure

1. Identify the critical paths that must never break.

2. Encode them as fast, deterministic tests.
 3. Keep a smoke subset that runs on every deploy.
 4. Grow the suite from every real incident — bug first, then test.
 5. Prune flaky and duplicate tests on a cadence.
-

Test Strategy

- **Type** — skill
- **Category** — qa #skill/qa
- **Trigger** — test strategy, how should we test, test plan, what tests do we need
- **Aliases** — test plan, testing approach, coverage strategy
- **Note** — test-strategy.md (in the zip package)

Design test strategies and test plans — coverage levels, approach, tooling, and what to test where.

Procedure

1. Identify what is being tested and the real risk surface.
 2. Choose the levels (unit / integration / e2e) and the shape of the pyramid for THIS system.
 3. Define coverage targets and the critical paths that must never break.
 4. Specify tooling, fixtures, and the test-data approach.
 5. Write the plan with explicit entry and exit criteria.
-

HR / People

Interview Kit

- **Type** — skill
- **Category** — hr #skill/hr
- **Trigger** — interview kit, create interview questions, interview guide, scorecard
- **Aliases** — interview guide, interview questions, scoring rubric
- **Note** — interview-kit.md (in the zip package)

Create structured interview kits with competency-based questions, STAR probes, and scoring rubrics.

Procedure

1. Define the competencies the role genuinely needs (not the wish list).
 2. Write behavioural questions with STAR follow-up probes.
 3. Build a scoring rubric anchored to observable signals.
 4. Assign questions across stages so nothing is asked twice.
 5. Add red-flag signals and what they map to.
-

Offer Analysis

- **Type** — skill
- **Category** — hr #skill/hr
- **Trigger** — prepare an offer, what should we offer, offer letter, comp for this offer
- **Aliases** — offer letter, comp benchmark, total compensation
- **Note** — offer-analysis.md (in the zip package)

Prepare a complete offer — comp benchmarking, internal equity check, total comp breakdown, and a draft offer letter.

Procedure

1. Benchmark the role against current market data.
 2. Check internal equity against the existing team at the same level.
 3. Build the total package: base, equity, bonus, sign-on.
 4. Draft the offer letter with the exact terms.
 5. Prepare negotiation guidance — walk-away, target, and the levers you will actually use.
-

Onboarding

- **Type** — skill
- **Category** — hr #skill/hr
- **Trigger** — onboarding plan, onboarding checklist, first-week plan, new hire setup
- **Aliases** — new hire onboarding, 30 60 90 plan, first week plan
- **Note** — onboarding.md (in the zip package)

Generate an onboarding checklist and first-week plan with 30/60/90-day goals.

Procedure

1. Build the pre-start list: accounts, equipment, buddy, first-day logistics.
 2. Schedule Day 1 and the full first week.
 3. Set 30/60/90-day goals jointly with the manager.
 4. Assign a buddy and a check-in cadence.
 5. Review formally at day 30 — do not let onboarding drift into month three.
-

People Report

- **Type** — skill
- **Category** — hr #skill/hr
- **Trigger** — headcount snapshot, attrition analysis, diversity metrics, org health report
- **Aliases** — headcount report, attrition report, org health
- **Note** — people-report.md (in the zip package)

Generate headcount, attrition, diversity, and org-health reports for leadership.

Procedure

1. Pull the people data from the HR system of record.

2. Compute headcount, attrition, span of control, and tenure.
 3. Trend turnover by team, level, and tenure band.
 4. Flag flight risk and any single-person concentration.
 5. Present with the decision the data informs, not just the numbers.
-

Performance Review

- **Type** — skill
- **Category** — hr #skill/hr
- **Trigger** — performance review, self-assessment, manager review, calibration
- **Aliases** — perf review, self assessment, calibration prep
- **Note** — performance-review.md (in the zip package)

Structure a performance review — self-assessment, manager template, and calibration prep.

Procedure

1. Gather specific behavioural examples, not adjectives.
 2. Draft the self-assessment.
 3. Write the manager review with evidence per competency.
 4. Prep rating distribution and any promotion case for calibration.
 5. Convert every vague note into a specific, actionable example.
-

Engineering

Architecture Decision

- **Type** — skill
- **Category** — engineering #skill/engineering
- **Trigger** — architecture decision, ADR, system design, choose between technologies
- **Aliases** — adr, architecture record, design decision
- **Note** — architecture.md (in the zip package)

Create or evaluate an architecture decision record (ADR) with explicit, honest trade-offs.

Procedure

1. State the decision and the forces acting on it (constraints, scale, team, cost).
 2. List the options genuinely considered.
 3. Record the trade-offs honestly, including what you give up.
 4. State the decision, its status, and its consequences.
 5. Note the conditions that would reverse it.
-

Code Review

- **Type** — skill
- **Category** — engineering #skill/engineering

- **Trigger** — review this code, review before I merge, is this code safe, security review
- **Aliases** — pr review, code review, security review
- **Note** — code-review.md (in the zip package)

Review code changes for security, performance, and correctness before merge.

Procedure

1. Read the change's intent from the PR description first.
 2. Check correctness and the edge cases that are actually reachable.
 3. Check security: injection, authorization, secret handling, unsafe defaults.
 4. Check performance: N+1 queries, unbounded loops, missing indexes.
 5. Check error handling and test coverage of the new path.
 6. Report findings by severity, each with the evidence that proves it.
-

Debugging

- **Type** — skill
- **Category** — engineering #skill/engineering
- **Trigger** — why is this failing, not working, crash, error message, broken, worked yesterday
- **Aliases** — root cause, debug, troubleshoot
- **Note** — debugging.md (in the zip package)

Find the real root cause of a failure before changing anything — reproduce, locate evidence, confirm a mechanism.

Procedure

1. Reproduce it reliably — an unreproducible bug is not yet a bug you can fix.
 2. Locate the evidence: logs, traces, metrics, recent changes.
 3. Read the evidence literally; do not assume what it should say.
 4. Confirm a mechanism that fully explains the behaviour.
 5. Fix the cause, not the symptom, then verify the fix against the reproduction.
-

Incident Response

- **Type** — skill
- **Category** — engineering #skill/engineering
- **Trigger** — we have an incident, production is down, service degraded, postmortem
- **Aliases** — incident, outage, postmortem
- **Note** — incident-response.md (in the zip package)

Run an incident end to end — triage, communicate, mitigate, and write the postmortem.

Procedure

1. Triage severity from customer impact and urgency.
2. Name an incident commander and a single comms channel.
3. Communicate on a fixed cadence, even when there is no update.

4. Mitigate before diagnosing — rollback, flag off, drain.
 5. Write a blameless postmortem with owned, dated action items.
-

Supply Chain Security

- **Type** — skill
- **Category** — engineering #skill/engineering
- **Trigger** — supply chain security, pin actions, dependency audit, sbom, pin by sha
- **Aliases** — sbom, dependency audit, sha pinning, ci security
- **Note** — supply-chain-security.md (in the zip package)

Harden the build supply chain — SHA-pinned actions and images, SBOM, dependency auditing, and least privilege.

Procedure

1. Inventory every dependency, base image, and CI action.
 2. Require SHA-pinning for actions and container images — a tag is mutable and is not a pin.
 3. Generate an SBOM and scan it for known vulnerabilities on a schedule.
 4. Apply least privilege to CI tokens and app permissions.
 5. Gate merges on the audit, and record any exception with an owner and an expiry.
-

Productivity

Call Preparation

- **Type** — skill
- **Category** — productivity #skill/productivity
- **Trigger** — prep me for my call, prepare for a meeting, call prep brief
- **Aliases** — meeting prep, call brief, pre-call research
- **Note** — call-prep.md (in the zip package)

Prepare for a customer or prospect call using CRM and signal data.

Procedure

1. Pull CRM and signal data for the contact/account.
 2. Gather recent context and open items.
 3. Produce a call brief with talking points and goals.
-

Digest

- **Type** — skill
- **Category** — productivity #skill/productivity
- **Trigger** — morning digest, weekly summary, catch me up, what happened while I was out
- **Aliases** — daily digest, weekly summary, activity digest
- **Note** — digest.md (in the zip package)

Generate a daily or weekly digest of activity across all connected sources.

Procedure

1. Pull activity across connected sources.
 2. Summarize by category.
 3. Surface anything needing attention.
 4. Deliver a scannable digest.
-

Standup

- **Type** — skill
- **Category** — productivity #skill/productivity
- **Trigger** — standup, daily standup, what did I work on yesterday, prep my standup
- **Aliases** — daily standup, yesterday summary, dev update
- **Note** — standup.md (in the zip package)

Generate a standup update from recent activity.

Procedure

1. Gather recent commits, PRs, ticket moves.
 2. Structure into a scannable update.
 3. Include blockers and today's plan.
-

Task Management

- **Type** — skill
- **Category** — productivity #skill/productivity
- **Trigger** — add a task, mark task done, show my tasks, task list
- **Aliases** — tasks, to-do, task list
- **Note** — task-management.md (in the zip package)

Manage a personal task list stored in TASKS.md; add, complete, and review tasks.

Procedure

1. Read TASKS.md.
 2. Add/update/complete tasks.
 3. Keep the file in sync.
 4. Optionally tie to calendar/project tracker.
-

Media

Design Critique

- **Type** — skill
- **Category** — media #skill/media
- **Trigger** — review this design, critique this mockup, what do you think of this screen

- **Aliases** — design feedback, usability review, visual hierarchy
- **Note** — design-critique.md (in the zip package)

Get structured design feedback on usability, hierarchy, and consistency.

Procedure

1. Review the design against usability, hierarchy, consistency.
 2. Give specific, structured feedback.
 3. Prioritize by impact.
-

Media Generation

- **Type** — skill
- **Category** — media #skill/media
- **Trigger** — generate image, make an image, create a video, generate music, make a song, sound effect, voiceover, logo
- **Aliases** — image generation, video generation, music, voiceover, logo
- **Note** — media.md (in the zip package)

Generate images, videos, music, speech, and sound effects from descriptions, with image-editing flows.

Procedure

1. Infer the scenario and intended style.
 2. Sanitize the prompt (no brand/artist/copyright for video and music).
 3. Pick the right aspect ratio.
 4. Generate, then offer edits or an image-to-video flow.
-

Video Edit

- **Type** — skill
- **Category** — media #skill/media
- **Trigger** — edit this video, cut this video, add captions, make this a short, trim video
- **Aliases** — video captions, trim video, reel, jump cut
- **Note** — video-edit.md (in the zip package)

Edit existing videos — cut retakes and silences, captions, trim, crop, merge, speed, audio cleanup.

Procedure

1. Discover the environment and source clip.
 2. Cut via transcript for retakes/silences.
 3. Burn captions or trim/crop as requested.
 4. Verify before delivering.
-

Legal

Compliance Tracking

- **Type** — skill
- **Category** — legal #skill/legal
- **Trigger** — compliance, audit prep, SOC 2, ISO 27001, GDPR
- **Aliases** — soc 2, iso 27001, gdpr, audit readiness
- **Note** — compliance-tracking.md (in the zip package)

Track compliance requirements and audit readiness (SOC 2, ISO 27001, GDPR).

Procedure

1. Identify applicable requirements.
 2. Track evidence and controls.
 3. Flag gaps and deadlines.
 4. Prepare audit documentation.
-

Contract Review

- **Type** — skill
- **Category** — legal #skill/legal
- **Trigger** — review this contract, what am I signing, red flags, check the payment terms
- **Aliases** — contract review, nda review, msa, red flags
- **Note** — contract-review.md (in the zip package)

Lightweight NDA, MSA, and vendor contract review for SMBs — flags non-standard terms in plain English.

Procedure

1. Read the contract from file/upload/attachment.
 2. Flag non-standard terms.
 3. Explain risks in plain English.
 4. Output a severity-tiered review with a negotiation playbook.
-

Risk Assessment

- **Type** — skill
- **Category** — legal #skill/legal
- **Trigger** — what are the risks, risk assessment, risk register, what could go wrong
- **Aliases** — risk register, mitigation plan, risk analysis
- **Note** — risk-assessment.md (in the zip package)

Identify, assess, and mitigate operational risks.

Procedure

1. Enumerate risks for the project/vendor/decision.

2. Score by likelihood and impact.
 3. Propose mitigations.
 4. Build a prioritized risk register.
-